

Programming Assignment

Graphical Summaries and Data Wrangling

In this assignment you will apply some basic Pandas tools to perform graphical summaries on the `Boulder Weather Data`. This will be the same data set from the previous assignment.

Once you have completed the assignment, you will need to download as a pdf and upload to the subsequent peer review for grading.

First, you will load Numpy and Pandas using their common aliases, `np` and `pd`.

```
In [1]: import numpy as np
import pandas as pd
```

You will also need to load `Matplotlib's PyLab library` and to set up Jupyter so that it will plot directly in the notebook.

```
In [2]: import matplotlib.pyplot as plt
%matplotlib inline
```

First, load the weather data into a Pandas DataFrame using `read_csv()`.

```
In [3]: # Two different paths to the data
local_path = 'clean_boulder_weather.csv'

# Load the data into a DataFrame
dfw = pd.read_csv(local_path)

# Inspect some of the data
dfw.head()
```

```
Out[3]:
```

	STATION	NAME	DATE	PRCP	TMAX	TMIN
0	USW00094075	BOULDER 14 W, CO US	2017-07-01	0.00	68.0	31.0
1	USW00094075	BOULDER 14 W, CO US	2017-07-02	0.00	73.0	35.0
2	USW00094075	BOULDER 14 W, CO US	2017-07-03	0.00	68.0	46.0
3	USW00094075	BOULDER 14 W, CO US	2017-07-04	0.05	68.0	43.0
4	USW00094075	BOULDER 14 W, CO US	2017-07-05	0.01	73.0	40.0

Recall that the data has the following columns:

- **STATION**: The unique identification code for each weather station
- **NAME**: The location / name of the weather station
- **DATE**: The date of the observation
- **PRCP**: The precipitation (in inches)
- **TMAX**: The daily maximum temperature (in Fahrenheit)
- **TMIN**: The daily minimum temperature (in Fahrenheit)

And recall that the data set contains data from several weather stations. Their names were

```
In [4]: station_names = list(set(dfw["NAME"]))
print("Station Names: ", station_names)
```

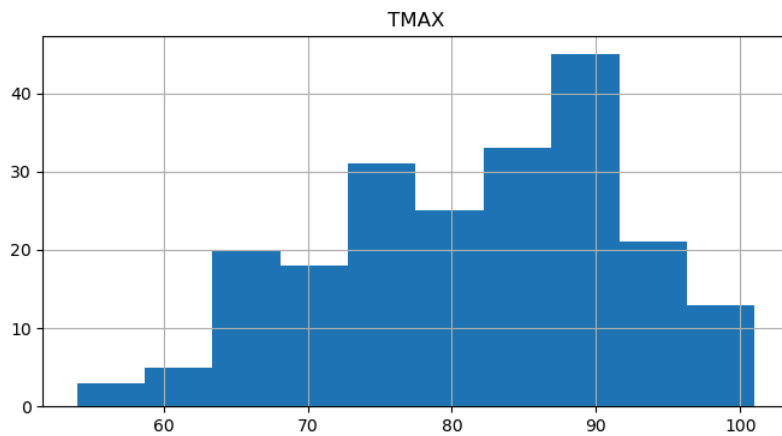
```
Station Names:  ['NORTHGLENN, CO US', 'BOULDER, CO US', 'BOULDER 14 W, CO US', 'SUGARLOAF COLORADO, CO US', 'RALSTON RESERVOIR, CO US', 'GROSS R
ESERVOIR, CO US', 'NIWOT, CO US']
```

Histograms of Maximum Temperature

If you desire to draw a histogram of the **TMAX** characteristics over all weather stations. There are several ways to plot histograms in Python. We will use Pandas built-in histogram function because it is designed to handle missing data well, but Matplotlib has a [native hist\(\)](#) function that you can use if you prefer. Furthermore, the Pandas `hist()` function actually calls the Matplotlib `hist()` function internally, so any parameters that work with Matplotlib's function should work with the Pandas function.

```
In [5]: # Initialize figure and axis
fig, ax = plt.subplots(figsize=(8,4))

# Plot histogram
dfw.hist(column="TMAX", ax=ax);
```



Here we've made a simple histogram of the **TMAX** data using Pandas' `hist()` function. Notice that we call the `hist()` function on the entire DataFrame and then indicate which column we want to use using the `column` option. We then pass in the figure axis to the function so that we can make modifications to the layout later on.

Pandas' automatic bin selection is reliable, however if you desire to define your own bins you can do so by passing in an optional `bins` argument to `hist()`.

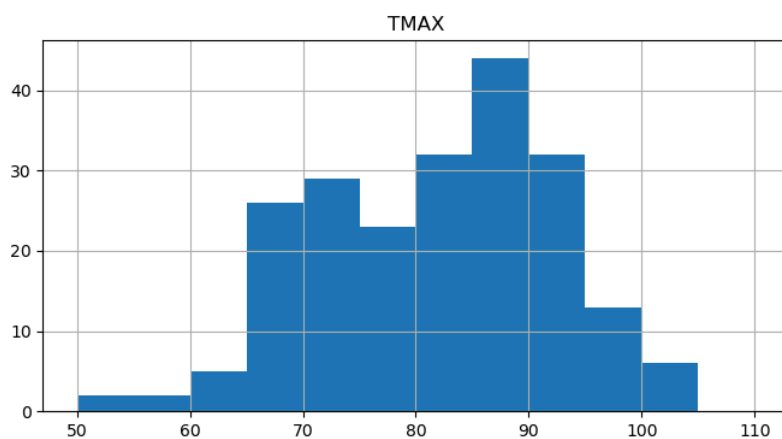
Both options are to pass in an integer value for the number of bins you want or a list specifying the bin edges. Suppose for example we want bins of width 5 starting at 50 and ending at 110. We could do the following:

```
In [6]: my_bins = range(50,115,5)
print("bin edges = ", list(my_bins))

bin edges = [50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, 110]
```

```
In [7]: # Initialize figure
fig, ax = plt.subplots(figsize=(8,4))

# Plot histogram
dfw.hist(column="TMAX", ax=ax, bins=my_bins);
```



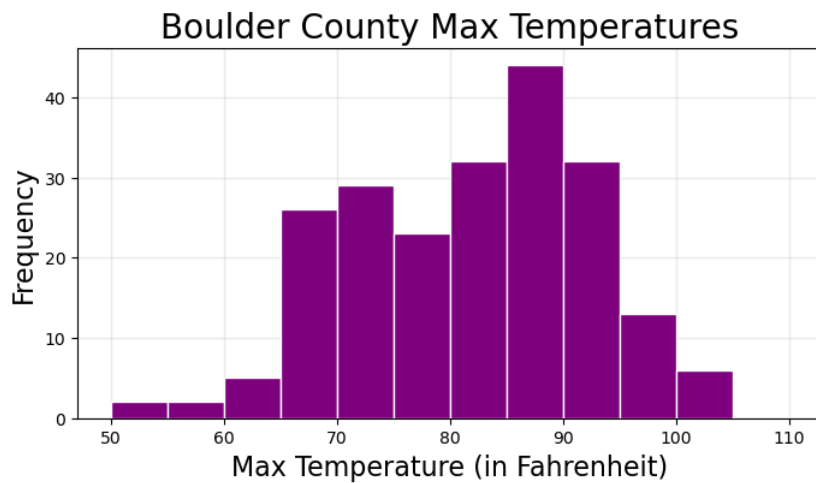
```
In [8]: # Initialize figure
fig, ax = plt.subplots(figsize=(8,4))

# Plot histogram with custom colors
dfw.hist(column="TMAX", ax=ax, bins=my_bins, facecolor="purple", edgecolor="white")

# Add a title
ax.set_title("Boulder County Max Temperatures", fontsize=20)

# Add axis labels
ax.set_xlabel("Max Temperature (in Fahrenheit)", fontsize=16)
ax.set_ylabel("Frequency", fontsize=16)

# Make the grid lines lighter and put them behind data
ax.grid(alpha=0.25)
ax.set_axisbelow(True)
```



Dide-by-side or stacked histograms are a good way to compare two sets of data. Let's create histograms of **TMAX** for data from two different weather stations and stack them on top of each other. Note that in order to make a true comparison it's a good idea to set the range on the horizontal and vertical axes to be the same on both plots. For this experiment we'll use the data from the `NORTHGLENN` and `GROSS RESERVOIR` weather stations.

```
In [9]: # Initialize figure subplots
fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(8,8))

# -----
# Plot histogram for Northglenn on Top
# -----
dfw.loc[dfw["NAME"]=="NORTHGLENN, CO US"].hist(column="TMAX", ax=axes[0], facecolor="purple", edgecolor="white")

# Add titles and Labels
axes[0].set_title("Northglenn Max Daily Temperatures", fontsize=20)
axes[0].set_xlabel("Max Temperature (in degrees F)", fontsize=16)
axes[0].set_ylabel("Frequency", fontsize=16)

# Make grid lighter and set behind data
axes[0].grid(alpha=0.25)
axes[0].set_axisbelow(True)

# -----
# Plot histogram for SugarLoaf on Bottom
# -----
dfw.loc[dfw["NAME"]=="GROSS RESERVOIR, CO US"].hist(column="TMAX", ax=axes[1], facecolor="black", edgecolor="white")

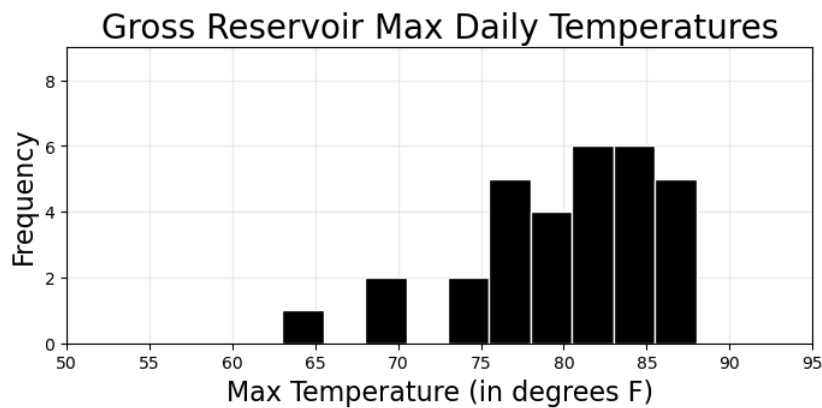
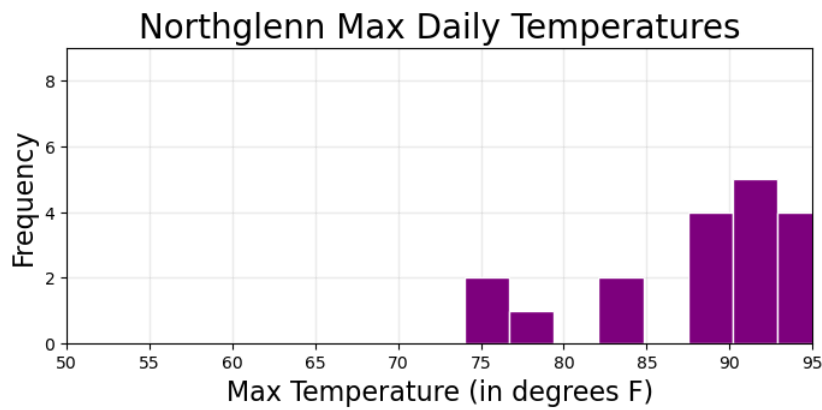
# Add titles and Labels
axes[1].set_title("Gross Reservoir Max Daily Temperatures", fontsize=20)
axes[1].set_xlabel("Max Temperature (in degrees F)", fontsize=16)
axes[1].set_ylabel("Frequency", fontsize=16)

# Make grid lighter and set behind data
axes[1].grid(alpha=0.25)
axes[1].set_axisbelow(True)

# -----
# Make the plots comparable
# -----

# Set x and y axis Limits to match
axes[0].set_xlim([50,95])
axes[1].set_xlim([50,95])
axes[0].set_ylim([0,9])
axes[1].set_ylim([0,9]);

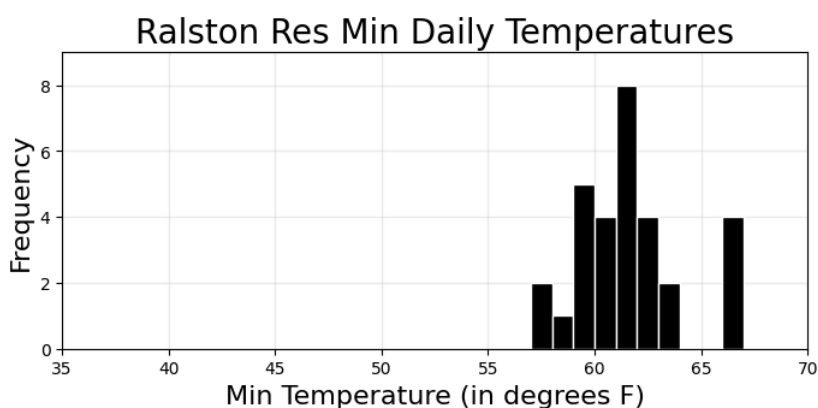
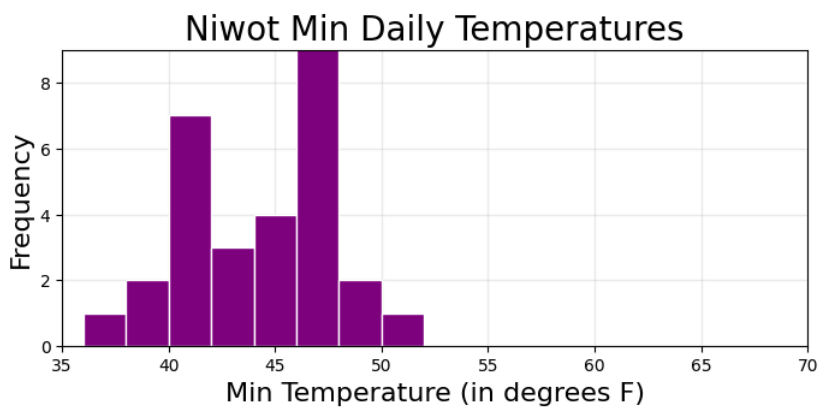
# Adjust vertical space so titles/axis labels don't overlap
fig.subplots_adjust(hspace=.5)
```



Observe that because we set the horizontal and vertical axis ranges to be the same we can easily make visual comparisons between the data. For instance it can be observed that it is generally warmer in the gross reservoir than in northglenn.

Exercise 1

Create stacked histograms comparing the minimum temperatures at `Niwot` and `Ralston Reservoir`. Feel free to adjust different parameters until you find a style and color scheme that you like. Don't forget to add meaningful titles and axis labels! Your histograms should look similar to the following:



```
In [10]: # Initialize figure subplots
fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(8,8))
```

```

#-----
# Plot histogram for Niwot on Top
#-----
dfw.loc[dfw["NAME"]=="NIWOT, CO US"].hist(column="TMIN", ax=axes[0], facecolor="purple", edgecolor="white")

# Add title and Labels
axes[0].set_title('Niwot Min Daily Temperatures', fontsize = 20)
axes[0].set_xlabel('Min Temperature', fontsize = 16)
axes[0].set_ylabel('Frequency', fontsize = 16)

# Make grid lighter and set behind the data
axes[0].grid(alpha=0.25)
axes[0].set_axisbelow(True)

#-----
# Plot histogram for Ralston Res on Bottom
#-----
dfw.loc[dfw["NAME"]=="RALSTON RESERVOIR, CO US"].hist(column="TMIN", ax=axes[1], facecolor="black", edgecolor="white")

# Add title and Labels
axes[1].set_title('Ralston Res Min Daily Temperatures', fontsize = 20)
axes[1].set_xlabel('Min Temperature', fontsize = 16)
axes[1].set_ylabel('Frequency', fontsize = 16)

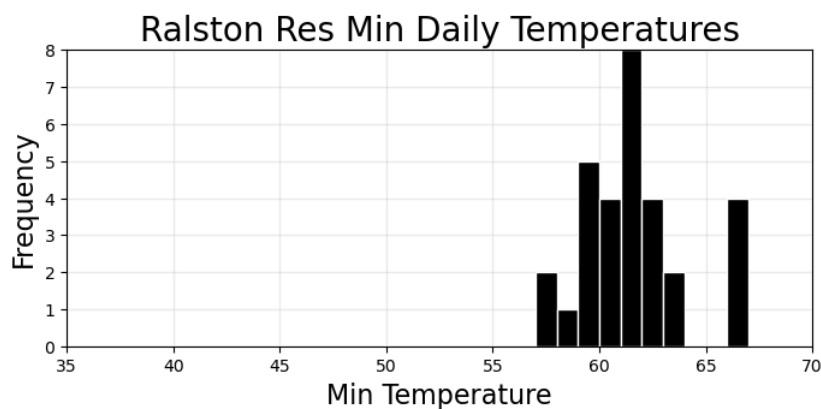
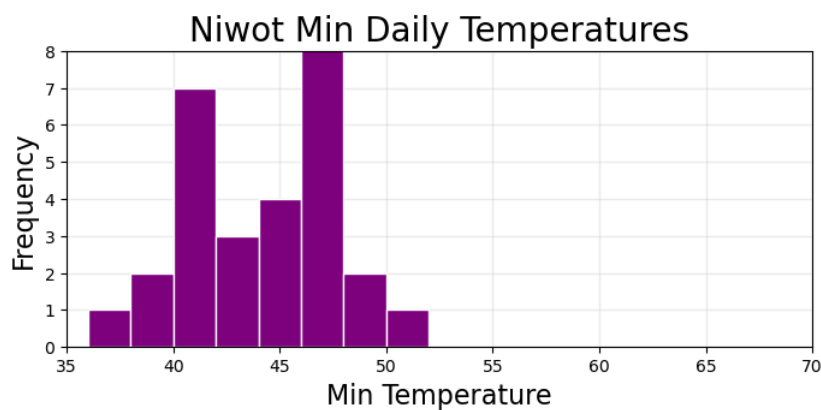
# Make grid lighter and set behind the data
axes[1].grid(alpha=0.25)
axes[1].set_axisbelow(True)

# Make the plots comparable

# Set x and y Labels to match
axes[0].set_xlim(35,70)
axes[1].set_xlim(35,70)
axes[0].set_ylim(0,8)
axes[1].set_ylim(0,8)

# Adjust vertical space so titles/axis labels don't overlap
fig.subplots_adjust(hspace=.5)

```



After you've gotten some decent *Frequency* histograms, pass the parameter **density=True** into the `hist()` function and redo the plots. Note that you'll probably have to change the vertical axis limits. How does the **density=True** parameter change the histograms?

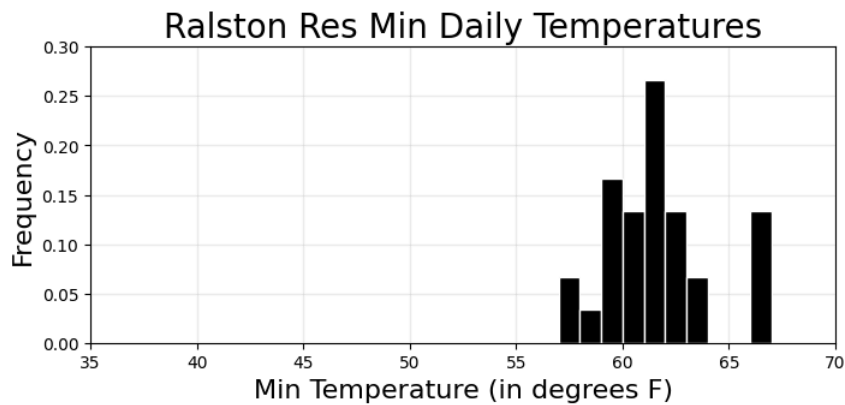
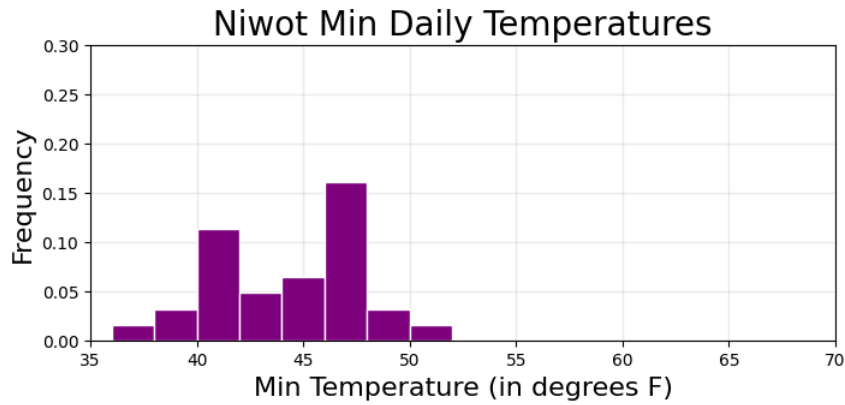
The following params could suffice:

```
axes[0].set_ylim([0,.3])
```

```
axes[1].set_ylim([0,.3]);
```

Note that in recent versions of Matplotlib, the `normed=True` parameter has been deprecated and replaced by the `density=True` parameter when creating histograms. This change was made to improve clarity and consistency in the library's API.

Your new histograms, should look similar to the following:



```
In [11]: # Initialize figure subplots
fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(8,8))

#-----
# Plot histogram for Niwot on Top
#-----
dfw.loc[dfw["NAME"]=="NIWOT, CO US"].hist(column="TMIN", ax=axes[0], density="True", facecolor="purple", edgecolor="white")

# Add title and Labels
axes[0].set_title('Niwot Min Daily Temperatures', fontsize = 20)
axes[0].set_xlabel('Min Temperature', fontsize = 16)
axes[0].set_ylabel('Frequency', fontsize = 16)

# Make grid lighter and set behind the data
axes[0].grid(alpha=0.25)
axes[0].set_axisbelow(True)

#-----
# Plot histogram for Ralston Res on Bottom
#-----
dfw.loc[dfw["NAME"]=="RALSTON RESERVOIR, CO US"].hist(column="TMIN", ax=axes[1], density = "True", facecolor="black", edgecolor="white")

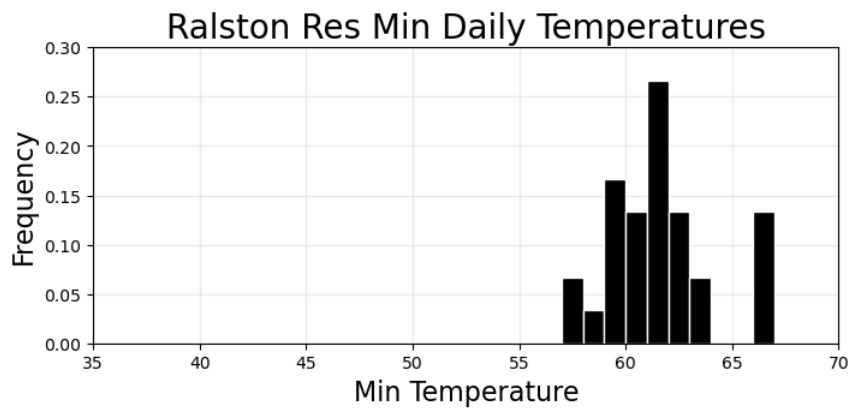
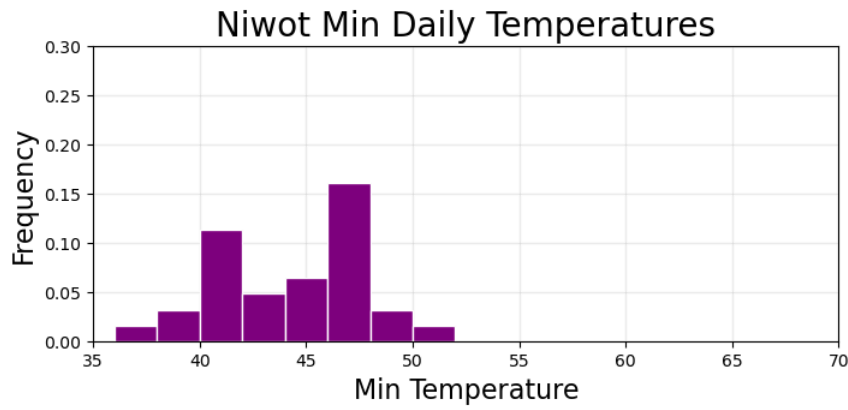
# Add title and Labels
axes[1].set_title('Ralston Res Min Daily Temperatures', fontsize = 20)
axes[1].set_xlabel('Min Temperature', fontsize = 16)
axes[1].set_ylabel('Frequency', fontsize = 16)

# Make grid lighter and set behind the data
axes[1].grid(alpha=0.25)
axes[1].set_axisbelow(True)

# Make the plots comparable

# Set x and y Labels to match
axes[0].set_xlim(35,70)
axes[1].set_xlim(35,70)
axes[0].set_ylim(0,.3)
axes[1].set_ylim(0,.3)

# Adjust vertical space so titles/axis labels don't overlap
fig.subplots_adjust(hspace=.5)
```



The Niwot minimum daily temperatures appear to have an approximately bimodal distribution, with temperatures ranging from about 36°F to 52°F. The Ralston Reservoir minimum daily temperatures have a more unimodal, roughly bell-shaped distribution, with most temperatures falling between 57°F and 64°F. Overall, Ralston Reservoir has higher minimum daily temperatures than Niwot.

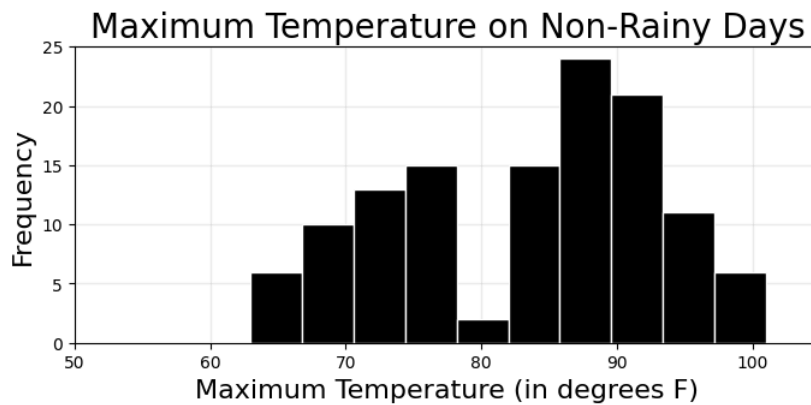
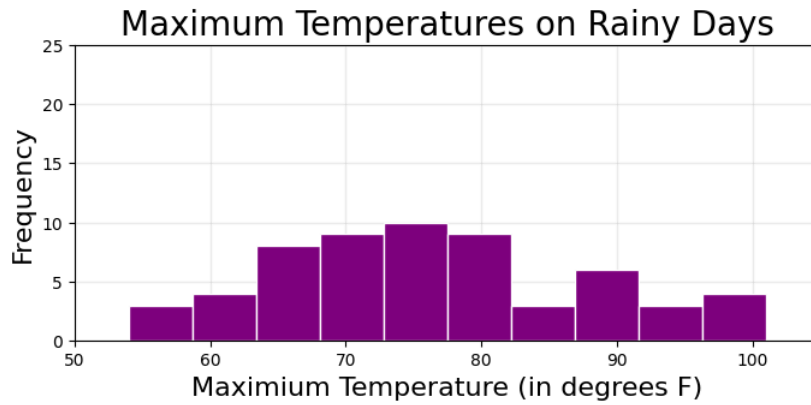
Exercise 2

Make stacked histograms to compare the maximum temperature over all of **Boulder County** on days when it rained vs days when it did not rain.

Hint: You will need to make the following arguments:

1. **Rainy Days:** `dfw.loc[dfw["PRCP"] > 0].hist ...etc`
2. **Non-Rainy Days:** `dfw.loc[dfw["PRCP"]==0].hist ...etc`
3. **Make sure to correctly title your plots as you have done in the previous plots with headings, and axis titles both in the x and in the y. There is no need to use the `density=true` function.**

Your histograms should look similar to the following:



```
In [12]: # Initialize figure subplots
fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(8,8))

#-----
# Plot histogram for Maximum Temperature on Rainy Days on Top
#-----
dfw.loc[dfw["PRCP"] > 0].hist(column="TMAX", ax=axes[0], facecolor="purple", edgecolor="white")

# Add title and Labels
axes[0].set_title('Maximum Temperature on Rainy Days', fontsize = 20)
axes[0].set_xlabel('Maximum Temperature (in degrees F)', fontsize = 16)
axes[0].set_ylabel('Frequency', fontsize = 16)

# Make grid lighter and set behind the data
axes[0].grid(alpha=0.25)
axes[0].set_axisbelow(True)

#-----
# Plot histogram for Maximum Temperature on Non-Rainy Days on Bottom
#-----
dfw.loc[dfw["PRCP"]==0].hist(column="TMAX", ax=axes[1], facecolor="black", edgecolor="white")

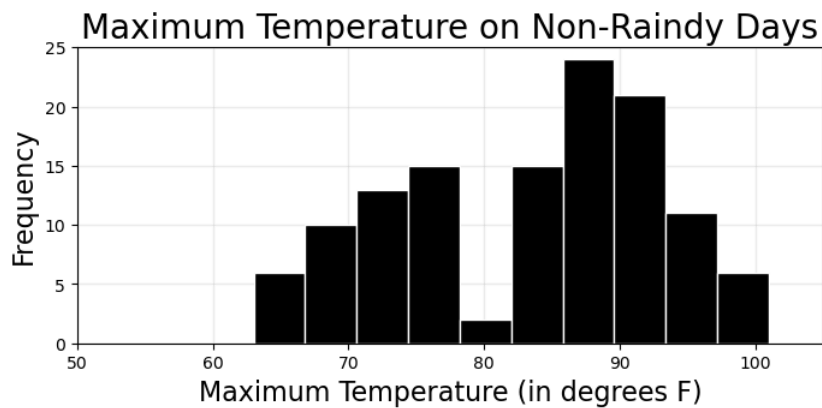
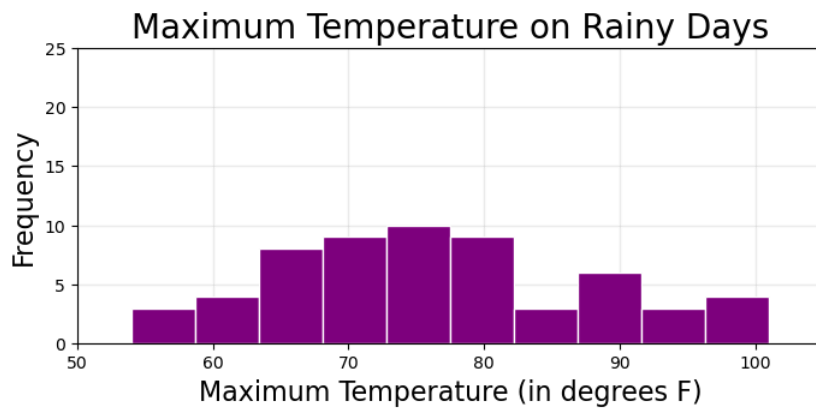
# Add title and Labels
axes[1].set_title('Maximum Temperature on Non-Rainy Days', fontsize = 20)
axes[1].set_xlabel('Maximum Temperature (in degrees F)', fontsize = 16)
axes[1].set_ylabel('Frequency', fontsize = 16)

# Make grid lighter and set behind the data
axes[1].grid(alpha=0.25)
axes[1].set_axisbelow(True)

# Make the plots comparable

# Set x and y Labels to match
axes[0].set_xlim(50,105)
axes[1].set_xlim(50,105)
axes[0].set_ylim(0,25)
axes[1].set_ylim(0,25)

# Adjust vertical space so titles/axis labels don't overlap
fig.subplots_adjust(hspace=.5)
```



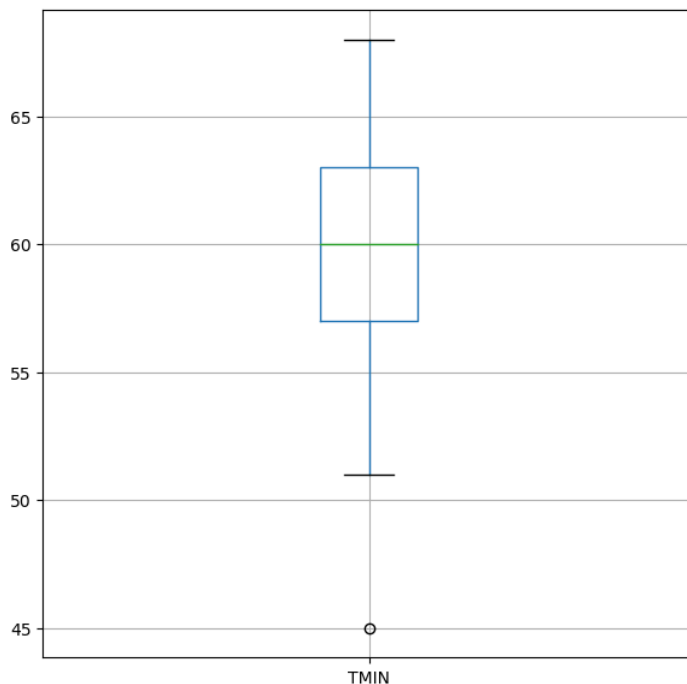
Box-and-Whisker Plots of Minimum Temperature

Next we'll look at Pandas' box-and-whisker plot functionality using the `boxplot()` function. We'll start by making a box-and-whisker plot for Northglenn's **minimum daily temperatures**. Like the `hist()` function, `boxplot()` is called on the entire DataFrame, and then we specify which column we're interested in using the `column` parameter.

```
In [13]: # Initialize figure
fig = plt.figure(figsize=(7,7))

# Get axis object
ax = fig.add_subplot(111)

# Plot histogram
dfw.loc[dfw["NAME"]=="NORTHGLENN, CO US"].boxplot(column="TMIN", ax=ax);
```



Note that the result is a standard box-and-whiskers plot with a box spanning the distance between the lower and upper quartiles, a (barely perceptible) median line, whiskers and caps, and outliers that fall more than $1.5 \times IQR$ outside of the nearest quartile.

Unfortunately, the default parameters don't lend themselves well to easy-to-read plots, so we'll make some minor adjustments.

```
In [14]: # Initialize figure
fig, ax = plt.subplots(figsize=(6,6))

# Plot histogram, but this time return dictionary of style parameters for modification after the fact
bp = dfw.loc[dfw["NAME"]=="NORTHGLENN, CO US"].boxplot(column="TMIN", ax=ax, widths=[.25], return_type='dict');

# -----
# Set properties of various parts of plot
# -----

# Change properties of boxes
for box in bp['boxes']:
    box.set(color='purple', linewidth=2)

# Change properties of whiskers
for whisker in bp['whiskers']:
    whisker.set(color='black', linewidth=2)

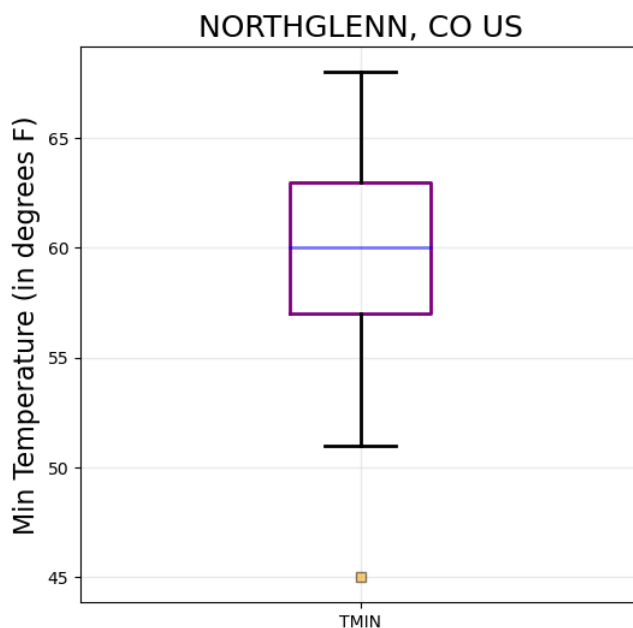
# Change properties of caps
for cap in bp['caps']:
    cap.set(color='black', linewidth=2)

# Change properties of median
for cap in bp['medians']:
    cap.set(color='blue', linewidth=2, alpha=0.5)

# Change properties of fliers (outliers)
for flier in bp['fliers']:
    flier.set(markerfacecolor='orange', linewidth=2, marker='s', markersize=6, alpha=0.5)

# Set title and vertical axis label
ax.set_title('NORTHGLENN, CO US', fontsize=18)
ax.set_ylabel("Min Temperature (in degrees F)", fontsize=16)

# Make grid-lines lighter
ax.grid(alpha=0.25)
```



Notice that the default settings for `boxplot()` gives us whiskers and an outlier. That is, the length of the whiskers are set equal to the largest distance between an observation and the upper or lower quartile that is less than $1.5 \times IQR$. Any data point beyond this distance is drawn as an outlier (or a *flier* as Pandas calls them).

Box-and-whisker plots are great for visualizing the spread of the data, outliers, and skew of the data. For instance, from this box-and-whisker plot we can tell that the minimum temperature at Northglenn has no skew because the the upper quartile Q_3 is about the same distance to the median as the lower quartile Q_1 is from the median.

Box-and-whisker plots are especially useful for comparing multiple sets of observations. For instance, the following side-by-side box-and-whisker plots compare the minimum temperature at Northglenn and Ralston Reservoir.

Note that the trick here is to select the rows of the DataFrame corresponding only to Northglenn and Ralston Reservoir and then call `boxplot()` with the `by` parameter to break the data into the two sets based on the value of the `NAME` column.

```
In [15]: # Initialize figure
fig, ax = plt.subplots(figsize=(6,8))

# To get just the columns corresponding to Northglenn and Ralston reservoirs, we
# select only the rows of the DataFrame corresponding to those names, then
# call boxplot and pass by="NAME" to break the data into the two sets
bp = dfw.loc[(dfw["NAME"]=="NORTHGLENN, CO US") | (dfw["NAME"]=="RALSTON RESERVOIR, CO US")].boxplot(
    column=["TMIN"], by="NAME", ax=ax, widths=[.5, .5], return_type="dict")

# This time, bp is a Pandas series containing multiple dictionaries
# corresponding to the two sets of data. We loop over each one individually
# and set the parameters we desire:
```

```

for column in bp:

    # Change properties in boxes
    for box in column['boxes']:
        box.set(color='purple', linewidth=2)

    # Change properties of whiskers
    for whisker in column['whiskers']:
        whisker.set(color='black', linewidth=2)

    # Change properties of caps
    for cap in column['caps']:
        cap.set(color='black', linewidth=2)

    # Change properties of median
    for cap in column['medians']:
        cap.set(color='orange', linewidth=2, alpha=0.5)

    # Change properties of fliers (outliers)
    for cap in column['fliers']:
        cap.set(markerfacecolor='orange', linewidth=2, marker='s', markersize=6, alpha=0.5)

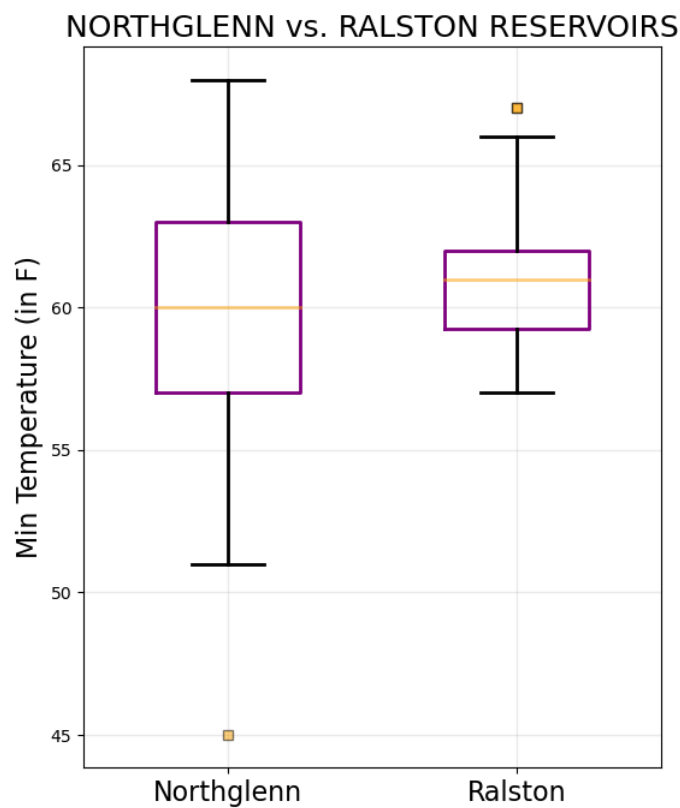
# Set title and vertical axis label
ax.set_title('NORTHGLENN vs. RALSTON RESERVOIRS', fontsize=18)
ax.set_ylabel("Min Temperature (in F)", fontsize=16)

# Set names of plots
plt.xticks([1,2],["Northglenn", "Ralston"], rotation=0, fontsize=16)

# Get rid of automatically generated titles and xlabels
plt.suptitle("")
ax.set_xlabel("")

# Make grid-lines lighter
ax.grid(alpha=0.25);

```

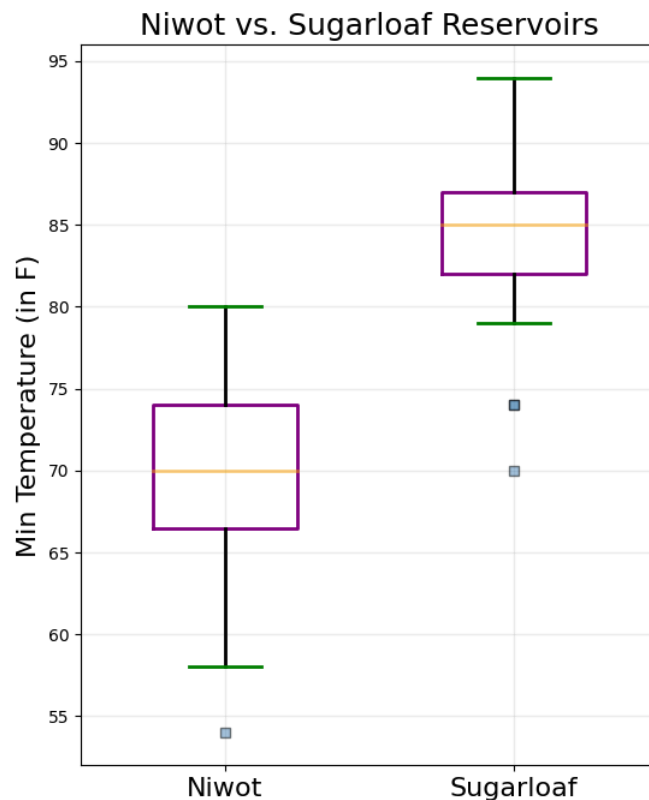


From the side-by-side box-and-whiskers plots we can easily see several things

- The median min daily temperature at **Ralston Reservoir** is more than 1 degrees warmer than that at **Northglenn**. Not too much of a difference here
- The min daily temperature at **Northglenn** does not really have a skew, while the min daily temperature at **Ralston Res** is slightly skewed. Do you think this is positively skewed or negatively skewed? Research the whisker plots!
- While the overall spreads of the two data sets are comparable, there is some slight but not too much variability at the **Ralston Reservoir** than at **Northglenn**.

Exercise 3

Plot side-by-side box and whisker plots of the max daily temperature in **Niwot** and **Sugarloaf**. Comment on any similarities and differences that you see in the distributions of the data. You may use the above exercise as a guide! It is highly encouraged.



Your plot should look similar to the following:

```
In [16]: # Initialize figure
fig, ax = plt.subplots(figsize=(6,8))

# To get just the columns corresponding to Niwot and Sugarloaf reservoirs, we
# select only the rows of the DataFrame corresponding to those names, then
# call boxplot and pass by="NAME" to break the data into the two sets
bp = dfw.loc[(dfw["NAME"]=="NIWOT, CO US") | (dfw["NAME"]=="SUGARLOAF COLORADO, CO US")].boxplot(
    column=["TMAX"], by="NAME", ax=ax, widths=[.5, .5], return_type="dict")

# This time, bp is a Pandas series containing multiple dictionaries
# corresponding to the two sets of data. We loop over each one individually
# and set the parameters we desire:
for column in bp:

    # Change properties in boxes
    for box in column['boxes']:
        box.set(color='purple', linewidth=2)

    # Change properties of whiskers
    for whisker in column['whiskers']:
        whisker.set(color='black', linewidth=2)

    # Change properties of caps
    for cap in column['caps']:
        cap.set(color='green', linewidth=2)

    # Change properties of median
    for cap in column['medians']:
        cap.set(color='orange', linewidth=2, alpha=0.5)

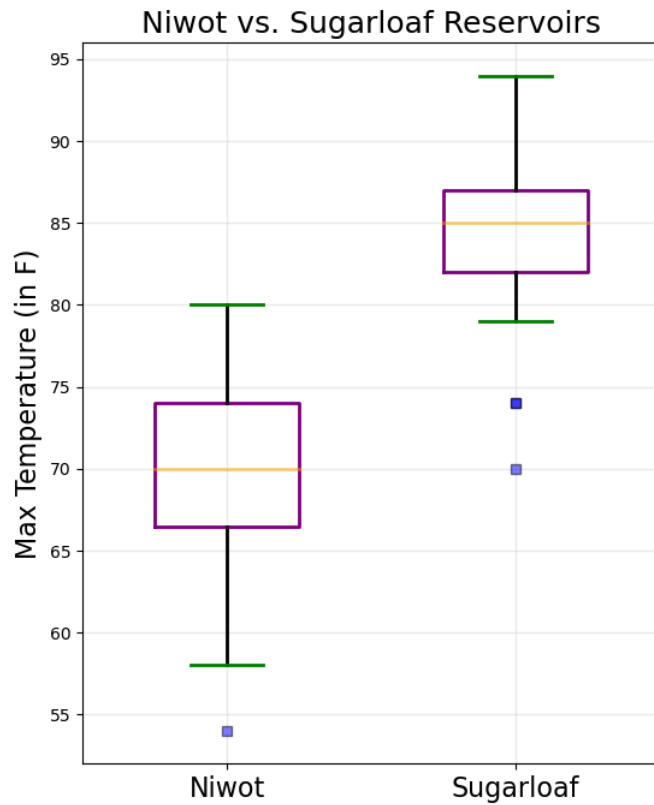
    # Change properties of fliers (outliers)
    for cap in column['fliers']:
        cap.set(markerfacecolor='blue', linewidth=2, marker='s', markersize=6, alpha=0.5)

# Set title and vertical axis label
ax.set_title('Niwot vs. Sugarloaf Reservoirs', fontsize=18)
ax.set_ylabel("Max Temperature (in F)", fontsize=16)

# Set names of plots
plt.xticks([1,2],["Niwot", "Sugarloaf"], rotation=0, fontsize=16)

# Get rid of automatically generated titles and xlabels
plt.suptitle("")
ax.set_xlabel("")

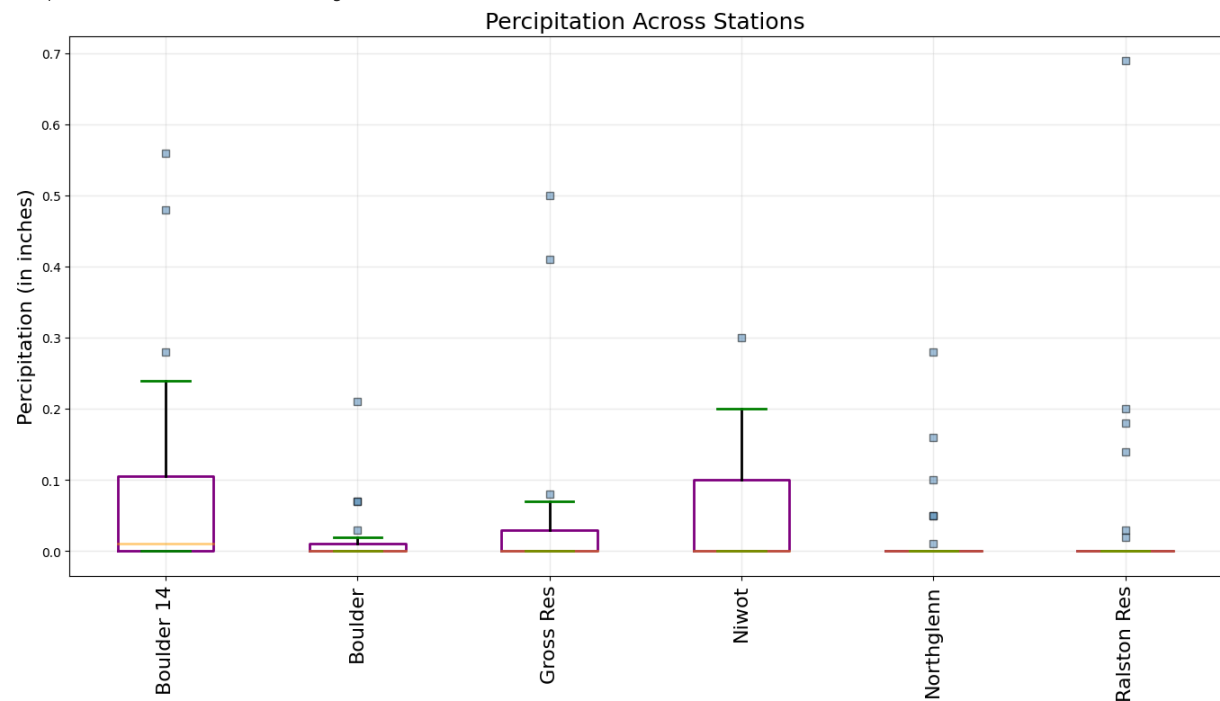
# Make grid-lines lighter
ax.grid(alpha=0.25);
```



Exercise 4: Putting it all together

Plot side-by-side box and whisker plots of the precipitation measured at **all** weather stations where we have precipitation data (**hint**: it's not all of them. **You will have to decide which one will be excluded!**). Mess with the plot parameters until you get a reasonable graphic that doesn't look too cramped. Note that it might be helpful to make the figure much wider and rotate the horizontal labels 90 degrees.

Your plot should look similar to the following:



```
In [17]: # Find which ones stations have precipitation
dfw.groupby("NAME")["PRCP"].count()
```

```
Out[17]: NAME
BOULDER 14 W, CO US      31
BOULDER, CO US          29
GROSS RESERVOIR, CO US  31
NIWOT, CO US            31
NORTHGLENN, CO US       30
RALSTON RESERVOIR, CO US 30
SUGARLOAF COLORADO, CO US 0
Name: PRCP, dtype: int64
```

```
In [18]: # Initialize figure
fig, ax = plt.subplots(figsize=(16,6))

# Plot weather stations that have precipitation data
bp=dfw.loc[dfw["NAME"]!="SUGARLOAF COLORADO, CO US"].boxplot(column="PRCP", by="NAME", ax=ax, widths=0.5,return_type = "dict")

# Loop over each one individually and set the parameters we desire:
for column in bp:

    # Change properties in boxes
    for box in column['boxes']:
        box.set(color='purple', linewidth=2)

    # Change properties of whiskers
    for whisker in column['whiskers']:
        whisker.set(color='black', linewidth=2)

    # Change properties of caps
    for cap in column['caps']:
        cap.set(color='green', linewidth=2)

    # Change properties of median
    for cap in column['medians']:
        cap.set(color='orange', linewidth=2, alpha=0.5)

    # Change properties of fliers (outliers)
    for cap in column['fliers']:
        cap.set(markerfacecolor='blue', linewidth=2, marker='s', markersize=6, alpha=0.5)

# Set title and vertical axis label
ax.set_title("Precipitation Across Stations", fontsize=18)
ax.set_ylabel("Precipitation (in inches)", fontsize=16)

# Set names of plots
plt.xticks([1,2,3,4,5,6],["Boulder 14", "Boulder", "Gross Res", "Niwtot", "Northglen", "Ralston Res"], rotation=90, fontsize=16)

# Get rid of automatically generated titles and xlabels
plt.suptitle("")
ax.set_xlabel("")

# Make grid-lines lighter
ax.grid(alpha=0.25);
```

